Week 3 Module

Topic 1 : Merge Sort

Question 1 (Easy) - Rank Student by Marks

A school has recently conducted a scholarship examination for all students of Class 12. The examination department has collected the marks of all students in an array. Due to the large number of participants, manually arranging marks in ascending order has become difficult.

Your task is to write a program that takes the marks of N students and sorts them in ascending order using the Merge Sort Algorithm.

Merge Sorts works by repeatedly dividing the array into smaller subarrays until each subarray contains a single element. These smaller subarrays are then merged into sorted manner to produce the final sorted array.

The school administration wants the marks to be displayed in increasing order so they can easily identify the lowest and highest scoring students and generate performance reports.

Input

- First line contains an integer N, representing the number of students
- Second line contains N space-separated integers, representing the marks of the students.

Output

- Print the marks in ascending order.

Constraints

$1 \leq N \leq 10^5$

$0 \leq Marks \leq 100$

Example

Input:

6

78 45 92 34 67 81

Output:

34 45 67 78 81 92

Solution

```
#include<iostream>
#include<vector>
using namespace std;
void merge(vector<int>&Marks, int st, int mid, int end){
vector<int> temp;
int i = st ;
int j = mid+1;
while(i<=mid && j<=end){
if(Marks[i]<=Marks[j]){
temp.push_back(Marks[i]);
i++;
}
else{
temp.push_back(Marks[j]);
j++;
}
}
while(i<=mid){
```

[redacted]

```
temp.push_back(Marks[i]);
i++;
}
while(j<=end){
temp.push_back(Marks[j]);
j++;
}
for(int idx=0; idx<temp.size(); idx++){
Marks[st+idx] = temp[idx];
}
}
```

```
void mergesort (vector<int> &Marks, int st, int end) {
if (st<end) {
    int mid = st+(end-st)/2 ;
    mergesort (Marks, st, mid);
    mergesort (Marks, mid+1, end);
    merge (Marks, st, mid, end);
  }
}

int main() {
  int N;
  if (!(cin>>N))
     return 0;
  vector <int> Marks(N);
  for (int i= 0; i<N; i++) {
       if (!(cin >> Marks[i]))
            return 0;
  }
  mergesort (Marks, 0, N-1);
  for (int i=0 ; i<N ; i++) {
       cout << Marks[i] << " ";
  }
  cout << endl;
  return 0 ;
}
```

Output

```
6
78 45 92 34 67 81
34 45 67 78 81 92


=== Code Execution Successful ===
```

Date : / /
Page No.

Question 2 (Easy) - Organizing Product Prices

An e-commerce company stores product prices in the order they are added to the platform. Before publishing a seasonal sales report, company wants all product prices to be arranged in ascending order.

Since the number of products can be very large, an efficient sorting technique is required. Your task is to implement Merge Sort to arrange the prices from the cheapest product to most expensive product.

The program should divide the list into smaller parts, sort each part, and merge them back together while maintaining the correct order

Input

- First line contains integer N
- Second line contains N space-separated product prices.

Output

- Print the sorted prices in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq Price \leq 1000000$

Example

Input

5

2500 1500 4000 3000 1000

Output

1000 1500 2500 3000 4000

Solution

```
#include <iostream>
#include <vector>
using namespace std;
void merge(vector<int>&Price, int st,int mid, int end){
vector <int> temp;
int i = st;
int j = mid+1;
while (i<=mid && j<=end){
if (Price[i] <= Price[j]){
    temp.push_back(Price[i]);
    i++;
}
else{
    temp.push_back(Price[j]);
    j++;
}
}
while(i<=mid){
  temp.push_back(Price[i]);
  i++;
}
while(j<=end){
  temp.push_back(Price[j]);
  j++;
}
for(int idx=0; idx<temp.size(); idx++){
  Price[st+idx] = temp[idx];
}
}
```

```
void mergesort (vector<int> &Price, int st, int end){
    if (st<end){
        int mid = st+ (end-st)/2;
        mergesort(Price, st, mid);
        mergeSort(Price, mid+1, end);
        merge(Price, st, mid, end);
    }
}

int main(){
    int N;
    if(!(cin>>N))
      return 0;
    vector<int> Price(N);
    for (int i=0; i<N; i++){
      if (!(cin>> Price[i]))
          return 0;
    }
    mergesort (Price, 0, N-1);
    for (int i=0; i<N; i++) {
        cout << Price[i] << " ";
    }
    cout << endl;
    return 0;
}
```

Output

```
5
2500 1500 4000 3000 1000
1000 1500 2500 3000 4000


=== Code Execution Successful ===
```

Date : / /
Page No.

Question 3 (Medium) - Sort Employee Records by Salary

A multinational company has employee salary records stored in an array. Before generating annual reports the Human Resources department wants the salary recordsarranged in ascending order.

The company expects the number of employees to grow significantly over time, so an efficient sorting algorithm must be used. You are required to implement Merge Sort and display the salaries in increasing order.

Additionally after sorting, the company wants to know:

- The lowest salary.
- The highest salary.
- The difference between the highest and lowest salary.

Input

- First line contains integer N.
- Second line contains N space-separated salaries.

Output

- First line : Sorted salaries.
- Second line : Lowest Salary.
- Third line : Highest salary
- Fourth line : Salary difference

Constraints

- $1 \leq N \leq 100000$
- $1000 \leq Salary \leq 1000000$

Date : / /
Page No.

Example

Input :

7

45000 25000 70000 50000 30000 60000 90000

Output:

25000 30000 45000 50000 60000 70000 90000

Lowest Salary : 25000

Highest Salary : 90000

Difference : 65000

Solution

```
#include <iostream>
# include <vector>
using namespace std;
void merge (vector <int> &Salary, int st, int mid, int end){
    vector <int> temp;
    int i=st;
    int j= mid+1 ;
    while (i<= mid && j<=end) {
        if( Salary [i]<= Salary[j]) {
                temp.push_back (Salary [i]) ;
                i++ ;
        }
        else {
                temp.push_back (Salary[j]);
                j++ ;
        }
    }
    while ( i<= mid) {
        temp.push_back ( Salary [i]) ;
        i++ ;
    }
    while (j<=end) {
        temp.push_back (Salary [j]) ;
        j++ ;
    }
    for (int idx=0 ; idx <temp.size() ; idx++) {
        Salary [idx+st] = temp [idx] ;
        }
}
```

```
void mergesort (vector<int> &Salary, int st, int end){
    if (st<end){
        int mid = st+(end-st)/2;
        mergesort (Salary, st, mid);
        mergesort (Salary, mid+1, end);
        merge (Salary, st, mid, end);
    }
}

int main(){
    int N;
    if(!(cin>>N))
      return 0;
    vector <int> Salary(N);
    for(int i=0; i<N; i++){
        if (!(cin >> Salary[i]))
            return 0;
    }
    mergesort (Salary, 0, N-1);
    for(int i=0; i<N; i++){
        cout << Salary[i] << " ";
    }
    cout << endl;
    if (N>0){
            cout << "Lowest Salary: " << Salary[0] << endl;
            cout << "Highest Salary: " << Salary[N-1] << endl;
            cout << "Difference: " << Salary[N-1] - Salary[0] << endl;
    }
    return 0;
}
```

Output

```
7
45000 25000 70000 50000 30000 90000 60000
25000 30000 45000 50000 60000 70000 90000
Lowest Salary: 25000
Highest Salary: 90000
Difference: 65000


=== Code Execution Successful ===
```

Date : / /
Page No.

TOPIC 2 : QUICK SORT

Question 1 (Easy) - Race Timing Analysis

A sports academy records the completion times (in seconds) of runners participating in a race. The coach wants to arrange timings from fastest runner to the slowest runner.

To process large datasets efficiently, the academy has decided to use the Quick Sort Alogrithm. Your task is to sort the race timings in ascending order.

Input

- First line contains integer N.
- Second line contains N - space separated timings

Output

- Print the timing in ascending order.

Constraints

$1 \leq N \leq 100000$

$1 \leq Time \leq 10000$

Example

Input

6

55 49 62 58 45 51

Output

45 49 51 55 58 62

Solution

```
#include<iostream>
#include<vector>
using namespace std;
int partition(vector<int> &Time, int low, int high){
    int pivot = Time[high];
    int i = low-1;
    for(int j= low ; j<high ; j++){
        if (Time[j] <= pivot){
                i++;
                swap(Time[i], Time[j]);
        }
    }
    swap(Time[i+1], Time[high]);
    return i+1;
}

void quicksort(vector<int> & Time, int low, int high){
    if(low< high){
        int idx = partition(Time, low, high);
        quicksort(Time, low, idx-1);
        quicksort(Time, idx+1, high);
    }
}
```

```
int main(){
    int N;
    if(!(cin>> N))
      return0;
    vector <int> Time(N);
    for(int i=0; i<N; i++){
       if(!(cin>> Time[i]))
          return 0;
     }
     quicksort(Time, 0, Time.size()-1);
     for(int i=0; i<N; i++){
           cout<< Time[i]<< " ";
      }
      cout << endl;
      return0;
}
```

Output

```
6
55 49 62 58 45 51
45 49 51 55 58 62


=== Code Execution Successful ===
```

Date : / /
Page No.

Question 2 (Easy)- Organizing Library Book IDs

A library stores books using unique numbers. Due to years of additions and removals, the IDs are no longer arranged properly.

The librarian wants all IDs sorted in ascending order before migrating the data to a new management system. Implement Quick Sort to arrange the IDs efficiently.

Input:

- First line contains integer N.
- Second line contains N space-separated book IDs.

Output:

- Print the sorted IDs.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq ID \leq 10^9$

Example

Input:

5
1025 1001 1050 1010 1005

Output

1001 1005 1010 1025 1050

Solution

```
#include <iostream>
#include <vector>
using namespace std;
int partition(vector <int> &ID, int low, int high){
    int pivot = ID[high];
    int i = low-1;
    for (int j = low; j<high ;j++){
        if ( ID[j] <= pivot){
            i++;
            swap(ID[i], ID[j]);
        }
    }
    swap(ID[i+1], ID[high]);
    return i+1;
}
void quicksort (vector <int> &ID , int low, int high){
    if (low <high){
        int idx = partition (ID, low, high);
        quicksort (ID, low, idx-1);
        quicksort(ID, idx+1, high);
    }
}
```

```
int main(){
int N;
if(!(cin>>N))
 return 0;
vector<int> ID(N);
for(int i=0;i<N;i++){
   if(!(cin>>ID[i])
      return 0;
  }
  quicksort(ID,0,ID.size()-1);
  for(int i=0;i<N;i++){
     cout<< ID[i] << " ";
    }
    cout <<endl;
    return 0;
}
```

Output

```
5
1025 1001 1050 1010 1005

1001 1005 1010 1025 1050


=== Code Execution Successful ===
```

Question 3 (Medium) - Online Store Sales Ranking

An online marketplace tracks the daily sales made by different sellers. At the end of each week, management wants to identify the best-performing sellers.
You are given the total sales made by N sellers. Using Quick Sort, arrange the sales figures in descending order.

After sorting display:

- Sorted sales figures.
- Top 3 highest sales values
- Average of the top 3 sales values

Input

- First line contains integer N.
- Second line contains N space-separated sales values.

Output

- Sorted sales figures in descending order.
- Top 3 sales values.
- Average of top 3 sales.

Constraints

- $3 \leq N \leq 100000$
- $1 \leq Sales \leq 1000000$

Date : / /
Page No.

Example

Input:

6
1200 5000 3000 2500 7000 4500

Output:

7000 5000 4500 3000 2500 1200
Top 3: 7000 5000 4500
Average: 5500

Solution

```
#include<iostream>
#include <vector>
using namespace std;
int partition (vector<int> &Sales, int low, int high){
  int pivot = Sales[High];
  int i = low-1;
  for (int j=low; j<high; j++) {
      if (Sales[j]<=pivot){
          i++;
          swap(Sales[i], Sales[j]);
      }
  }
  swap(Sales[i+1], Sales[High]);
  return i+1;
}

void quicksort (vector<int> &Sales, int low, int high){
    if (low<high){
        int idx = partition (Sales, low, high);
        quicksort (Sales, low, idx-1);
        quicksort (Sales, idx+1, high);
    }
}
```

```
int main(){
  int N;
 if(!(cin>>N)){
     return 0;
  }
  vector<int> Sales(N);
  for(int i=0 ; i<N ;i++){
     if(! (cin>> Sales[i]))
        return 0;
   }
  quicksort ( Sales, 0, Sales.size()-1);
  for(int i= N-1 ; i>=0 ;i--) {
       Cout << Sales[i] << " ";
  }

  cout << endl;
  double sum=0;
  int count =0;
  cout << "Top 3 :";
  for (int i= N-1 ; i>=0 && count<3 ;i--){
       Cout << Sales[i]<<" ";
        sum+= Sales[i];
         count++;
  }
  cout << endl;
  if (count>0) {
     double average = sum/count;
     cout << "Average" << ": "
                          << average << endl;
  }
  return 0;

}
```

Output

```
6
1200 5000 3000 2500 7000 4500

7000 5000 4500 3000 2500 1200
Top 3:7000 5000 4500
Average: 5500


=== Code Execution Successful ===
```